

Insertion

CO2: AT-3

Name: Manoj M
Reg no: 192511060
course code: CSA0306
course Name: Data Structures

1. Insertion:

create a new node
Read Data

IF insert at beginning

new $\rightarrow$ next = head

head = new

ELSE IF insert at end THEN

IF head == NULL

head = new

ELSE

traverse to last node

ELSE (insert at position), THEN

traverse to (position - 1)

new $\rightarrow$ next = temp $\rightarrow$ next

temp $\rightarrow$ next = new

Display list

STOP

2. Deletion

IF delete from beginning THEN

temp = head

head = head → next

Delete temp

ELSE IF delete from the end THEN

traversal untill last node

remove last node.

ELSE

Traverse to (position + 1)

delete required node

Display list

STOP

3. search an element

read key

set position = 1

temp = head

WHILE temp != NULL

IF temp → data == key

Print position

STOP

Print

"Element not found"

STOP

4. Reverse a singly linked list.

START

prev = NULL

current = head

WHILE current != NULL

next = current → next

current → next = prev

prev = current

current = next

head = prev

Display list

STOP

5. Count number of nodes

Start

count = 0

temp = head

WHILE temp != NULL

count ++

temp = temp → next

END WHILE

Print count

stop

6) Merge two sorted linked list
start
compare first nodes of both lists
while both lists are not empty
 choose smaller node
 attach it to merged list
 move the list forward

ENDWHILE

Attach remaining nodes
display merged list
stop

7) Remove duplicates from sorted list

START

temp = head

WHILE temp $\rightarrow$ next $\neq$ NULL

 IF temp $\rightarrow$ data == temp $\rightarrow$ next $\rightarrow$ data
 delete duplicate node

ELSE

 temp $\rightarrow$ temp $\rightarrow$ next

ENDIF

ENDWHILE

display list
stop

1) circular linked list creation & traversal

start

create node

link last node to first node

temp = head

do

print temp → data

temp = temp → next

while temp != head

stop

9) doubly linked list (forward & backward traversal)

START

create doubly linked list

Forward traversal

temp = head

while temp != NULL

print temp → data

save last node

temp = temp → next

Endwhile

Backward traversal

temp = last

while temp != NULL

print temp → data

temp = temp → prev

ENDWHILE

10. Polynomial addition using linked list

start

create a first polynomial

create a second polynomial

WHILE both lists are not empty

IF exponents are equal

add coefficient

store result

ELSE IF exponent 1 > exponent 2

store first term

ELSE

store second term

MOVE corresponding pointers

END WHILE

Attach remaining terms

Display resultant polynomial

STOP.